

Entrenamiento de NN en la práctica

Valero Laparra



Training NN

Cosas que mirar

- **Karpathy, receta.**

<http://karpathy.github.io/2019/04/25/recipe/>

- **Ejemplos con posibles curvas de aprendizaje**

<https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/>

https://en.wikipedia.org/wiki/Learning_curve

<https://wandb.ai/mostafaibrahim17/ml-articles/reports/A-Deep-Dive-Into-Learning-Curves-in-Machine-Learning--Vmldzo0NjA1ODY0>

- **Blog, tips.**

<https://towardsdatascience.com/a-bunch-of-tips-and-tricks-for-training-deep-neural-networks-3ca24c31ddc8>

- **Stanford, cheatshset.**

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-deep-learning-tips-and-tricks>

- **“Neural Networks: Tricks of the Trade”, libro. Blog:**

<https://machinelearningmastery.com/neural-networks-tricks-of-the-trade-review/>

- **Curso de Andrew Ng**

<https://www.coursera.org/lecture/deep-neural-network/train-dev-test-sets-cxG1s>

Training NN

**Entrenar algoritmos de ML es un arte:
Entrenar NN es magia negra**

- Entrenar de forma totalmente automática no es posible
- Usar la “tecnología” sin entenderla es un **error**
- Hay errores visible y errores invisibles
Para evitarlos hay que **monitorizar** TODO.

RECETA

Antes de entrenar:

- 1) Datos
- 2) Objetivo (función de coste)
- 3) Define Baselines

Entrenamiento:

- 4) Búsqueda arquitectura: Modelo simple vs Modelo complejo
- 5) Hiperparámetros de entrenamiento
- 6) Entrenamiento final + tips

Monitorizar

- XAI [Curvas de aprendizaje]
- Carga computacional en las NN

RECETA

1) Datos

"La red es una versión comprimida de tus datos"

- **Inspecciona** los datos: ojo especial con los **outliers** y **clases desbalanceadas**.
- Cuidado con el **preprocesado**, elimina información.
La normalización a veces es buena y a veces mala.
Mejor si los datos van entre $[0,1]$ o $[-1,1]$, la computación será mas rápida.
- Cuantos mas datos mejor, piensa en usar:
data augmentation o modelos generativos (no te pases con los datos sintéticos)
- Haz bien la **partición** de los datos: train, validación, test, (development,.
Los de test al cajón.
Cuidado: mismatching train/test, mezcla datos distinta calidad (dev, sim...),...
- Mejor supervisado (no te fies del unsupervised).

INGMATERI GRIPE2S 50..



Strulea



anglusti



Apenles



Lmbiller



Spmeuter



nairiyra



Carrttyr



plania



caprlar

RECETA

2) Función de coste

”Es el objetivo de tu red”

- **Define** bien tu función de coste, o exige que te la **definan bien**.
- Ojo especial con los **outliers y clases desbalanceadas**.
- Entrenamos usando “Empirical risk minimization”:
depende de la distribución de tus datos.
- Cuidado con las funciones de coste especiales.



RECETA

3) Define Baselines

”Necesitamos saber que está bien y mal”

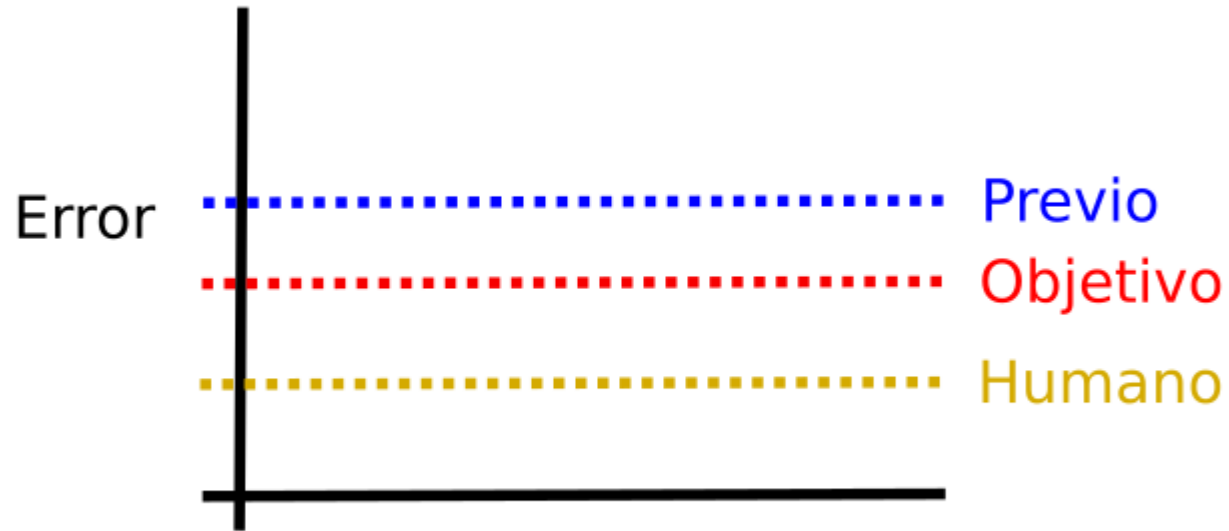
- Usa múltiples Baselines:

- ¿Qué es deseable?
- ¿Qué se ha hecho antes parecido?
- Coger al humano como gold standard.
- Valor de **la pérdida** al principio del entrenamiento.



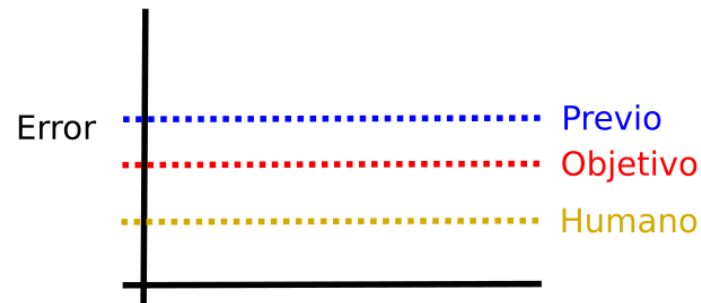
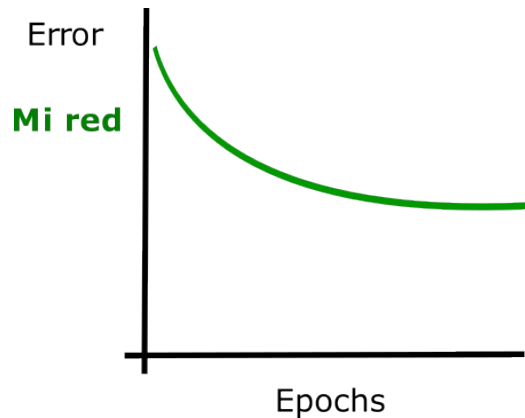
RECETA

3) Define Baselines



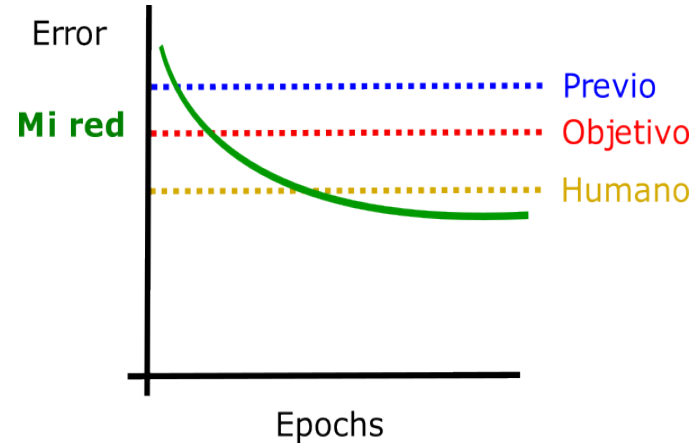
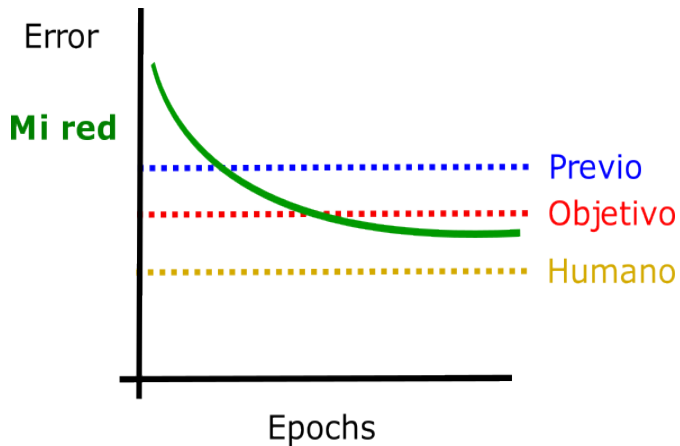
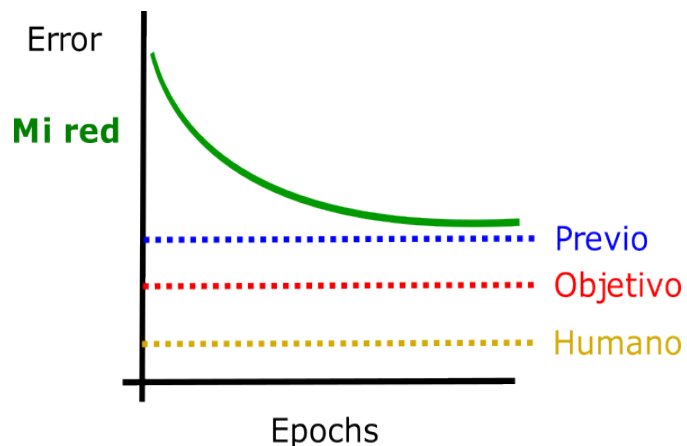
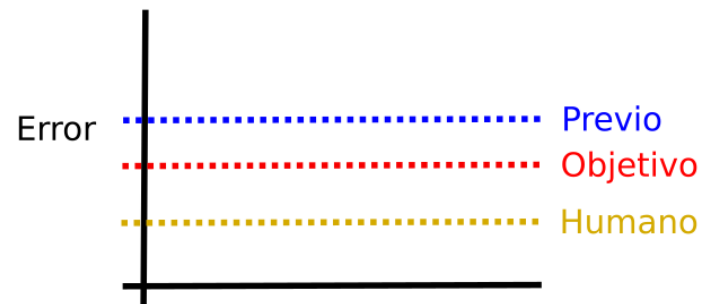
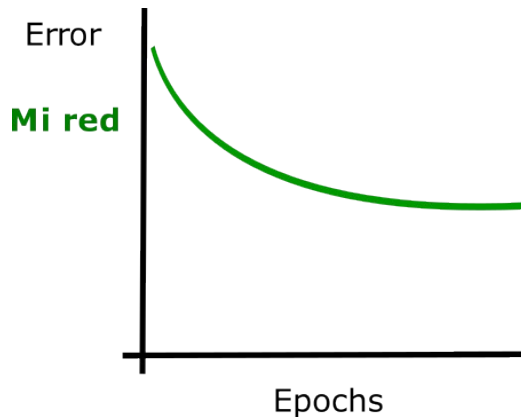
RECETA

3) Define Baselines



RECETA

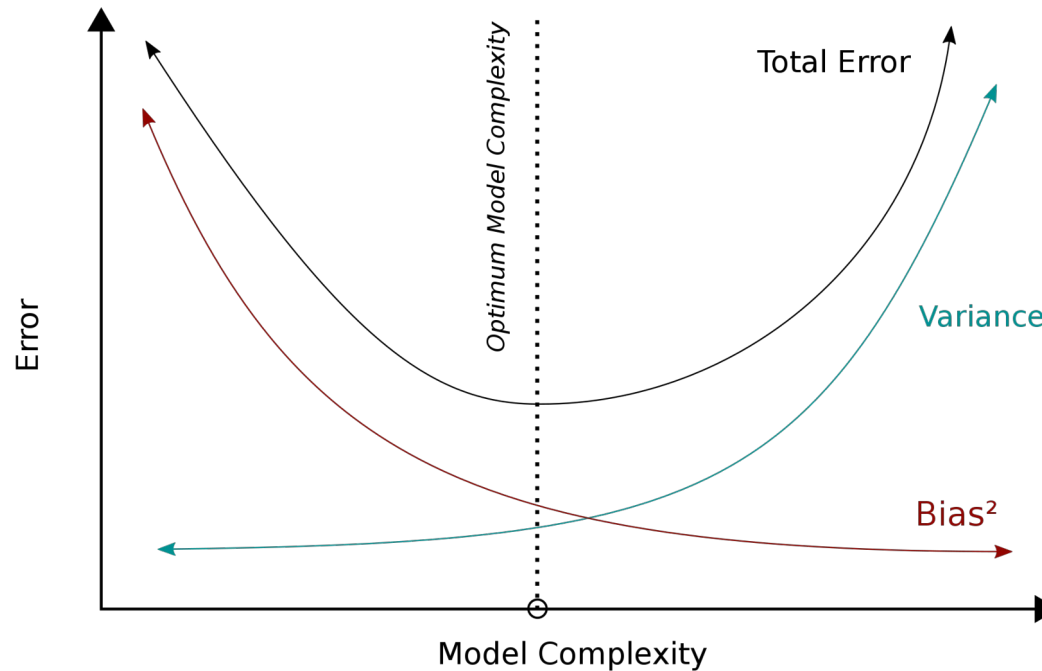
3) Define Baselines



RECETA

4) Búsqueda de la arquitectura.

Modelo Simple vs Modelo Complejo

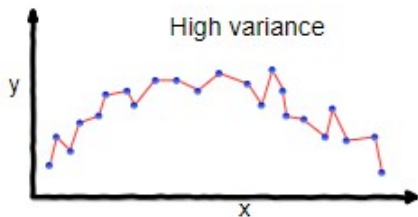


RECETA

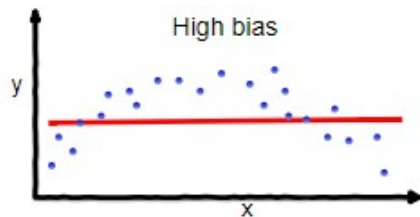
4) Búsqueda de la arquitectura. Evoluciona modelo simple y complejo

Minimizar bias (train) y el variance (validación)

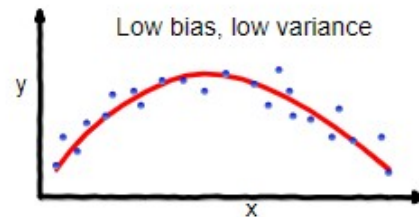
- Aumento complejidad → mejorar **train** (minimizo bias)
- Reduzco complejidad, (regularizo o mas datos) → mejorar **validación** (minimizo variance)



overfitting



underfitting



Good balance

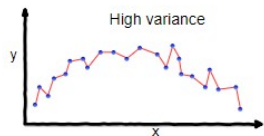
RECETA

4) Búsqueda de la arquitectura. Evoluciona modelo simple y complejo

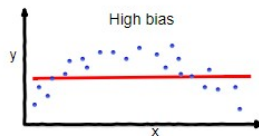
Minimizar bias (train) y el variance (validación)

- Aumento complejidad → mejorar **train** (minimizo bias)
- Reduzco complejidad, (regularizo o mas datos) → mejorar

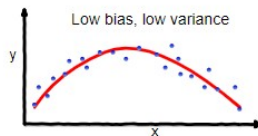
validación (minimizo variance)



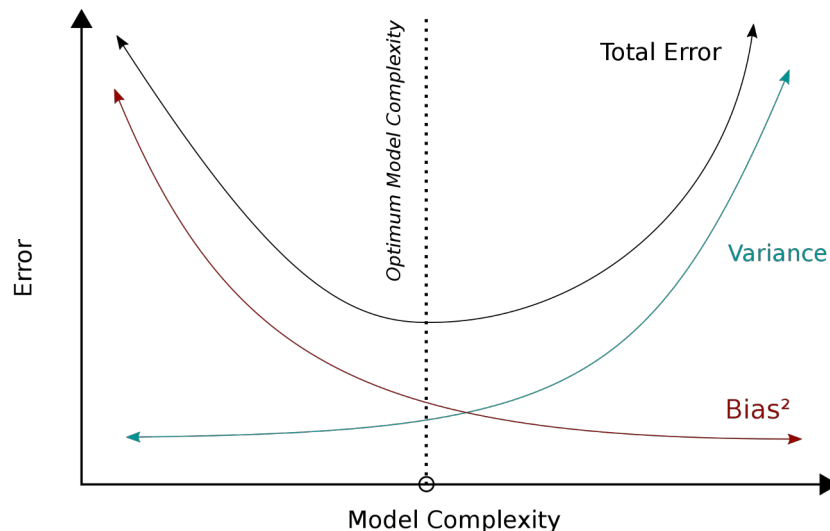
overfitting



underfitting



Good balance



RECETA

4) Búsqueda de la arquitectura. Evoluciona modelo simple y complejo

- Pasito a pasito:
 - Aumentamos tamaño modelo simple
 - Disminuimos tamaño modelo complejo
- Complejidad = Número de parámetros
 - Jerarquía (para reducir):**
Primero quitar neuronas, luego poner poolings, y luego quitar capas.
Primero capas densas, luego recurrentes y luego convolucionales.
- **AutoML o en grid**, por hacernos una idea.

RECETA

4) Búsqueda de la arquitectura.

Modelo Simple. Pruebas con el modelo mas tonto.

- **Simplifica:**
 - Modelo mas tonto, mejor empezar con modelo lineal.
 - Elimina lineas de código innecesarias.
- Usa **semilla** para inicializar.
- **Inicializa los pesos/activación de la última capa** acorde a tus datos (bias...).
- Para **explorar** no hacen falta muchísimas épocas.
- Complica el modelo **pasito a pasito**.



RECETA

4) Búsqueda de la arquitectura.

Modelo complejo (el otro extremo)

Busca un modelo complejo que aprenda mucho (incluso demasiado).
Sobreentrena en train y después regulariza.

- Modelo:

- **Grande** para que pueda sobreentrenar.
 - Busca lo que ha hecho la gente por ahí.
 - Usa redes preentrenadas.

- No muy loco, puedes coger uno **preentrenado**.
- Ten en cuenta cuantos datos tienes.

El número de parámetros ha de ser claramente menor que el número de datos.
(Al menos 10 veces menos)

- Disminuye la complejidad: **pasito a pasito**.



RECETA

4) Búsqueda de la arquitectura.

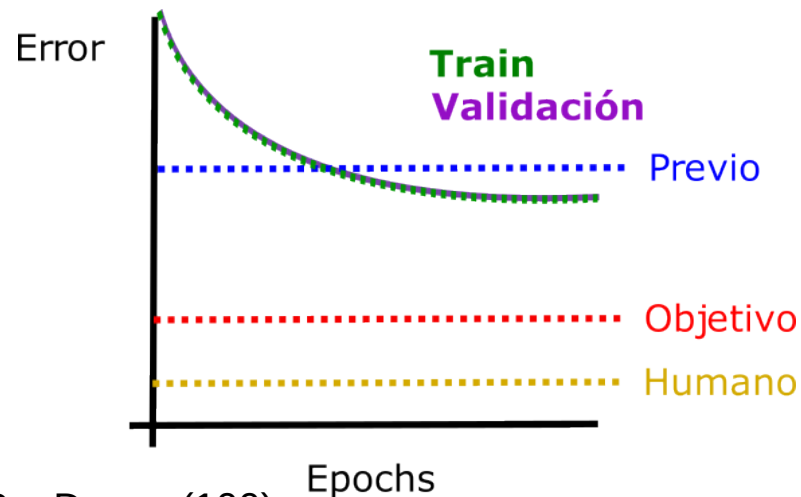
Ejemplo: Clasificación de imágenes

Datos: $X = (50.000, 32, 32, 3)$, $Y = (50.000, 100)$

Loss: crossentropy, o MSE

Arquitectura simple (mal train, similar validación):

- 1) Conv2D(100) + GAP
- 2) Conv2D(128) + GAP + Densa(100)
- 3) Conv2D(3,32) + Pooling(2) + Conv2D(3,128) + GAP + Densa (100)
- 4) Conv2D(3,64) + Pooling(2) + Conv2D(3,128) + GAP + Densa (100)
- 5) Conv2D(3,64) + Pooling(2) + Conv2D(3,128) + Pooling(2) + FL + Densa (100)
- 6) 3 (32,64,128) x (Conv2D(3) + Pooling(2)) + FL + Densa (100)



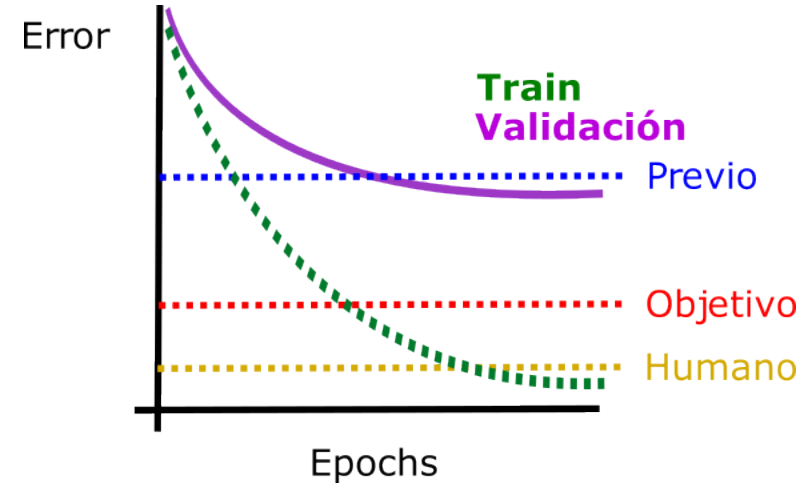
RECETA

4) Búsqueda de la arquitectura.

Ejemplo: Clasificación de imágenes

Datos: X = (50.000,32,32,3), Y = (50.000, 100)

Loss: crossentropy, o MSE



Arquitectura compleja (buen train, mal validación):

- 1) 4 (64,128,256,512) x (Conv2D(3) + Pooling(2)) + FL + Densa(1000) + Densa (100)
- 2) 4 (64,128,256,512) x (Conv2D(3) + Pooling(2)) + FL + Densa(200) + Densa (100)
- 3) 4 (64,128,256,512) x (Conv2D(3) + Pooling(2)) + FL + Densa (100)
- 4) 4 (64,128,256,256) x (Conv2D(3) + Pooling(2)) + FL + Densa (100)
- 5) 3 (64,128,256) x (Conv2D(3) + Pooling(2)) + FL + Densa (100)
- 6) 3 (64,128,128) x (Conv2D(3) + Pooling(2)) + FL + Densa (100)
- 7) 3 (64,64,128) x (Conv2D(3) + Pooling(2)) + FL + Densa (100)
- 8) 3 (32,64,128) x (Conv2D(3) + Pooling(2)) + FL + Densa (100)

RECETA

4) Búsqueda de la arquitectura.

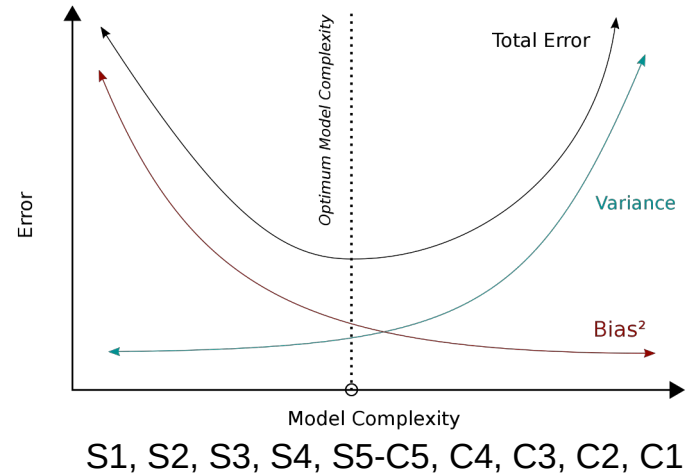
Ejemplo: Clasificación de imágenes

Datos: $X = (50.000, 32, 32, 3)$, $Y = (50.000, 100)$

Loss: crossentropy, o MSE

Y después regularizar S6-C8

3 (32,64,128) x (Conv2D(3) + Pooling(2)) + FL + Densa (100)



RECETA

5) Hiperparámetros de entrenamiento

- Número de épocas
Las suficientes para que esté convergido claramente.
Es decir, la curva de validación no mejora.
- Batch size
Tan grande como pueda.
Es el “causante principal” de la carga computacional.
- Regularización
- Learning rate



RECETA

5) Hiperparámetros de entrenamiento

- Número de épocas

Las suficientes para que esté convergido claramente.
Es decir, la curva de validación no mejora.

- Batch size

Tan grande como pueda.
Es el “causante principal” de la carga computacional.

- Regularización

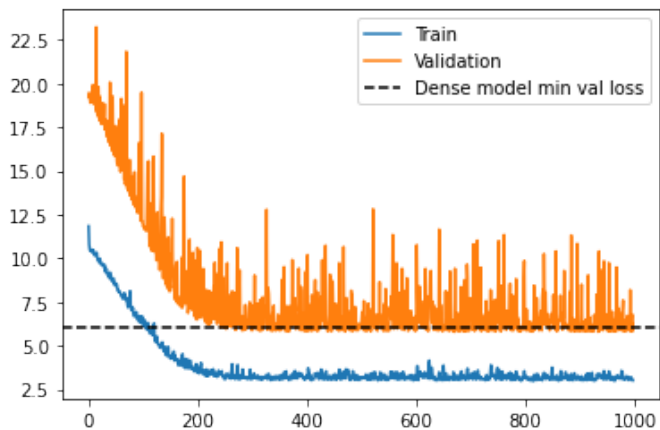
La necesaria.
Que la curva de validación y de train estén pegaditas.

- Learning rate

RECETA

5) Hiperparámetros de entrenamiento: regulariza

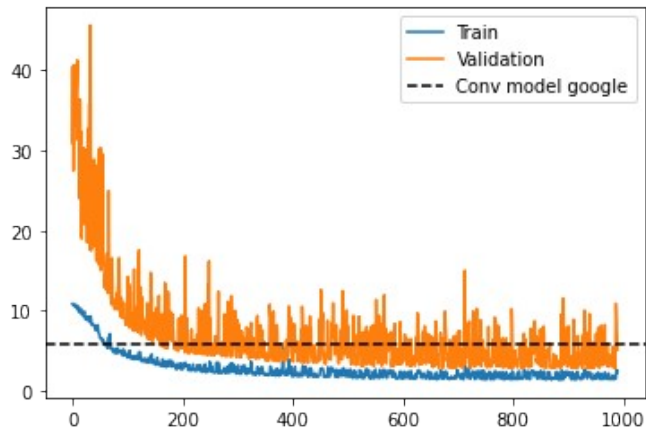
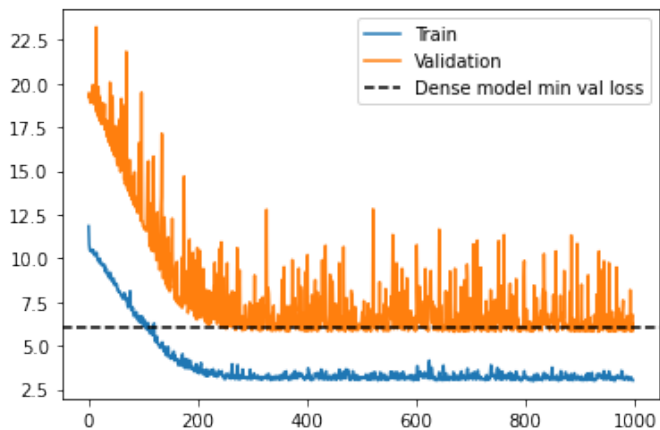
Vamos a ver que pasa en validación



RECETA

5) Hiperparámetros de entrenamiento: regulariza

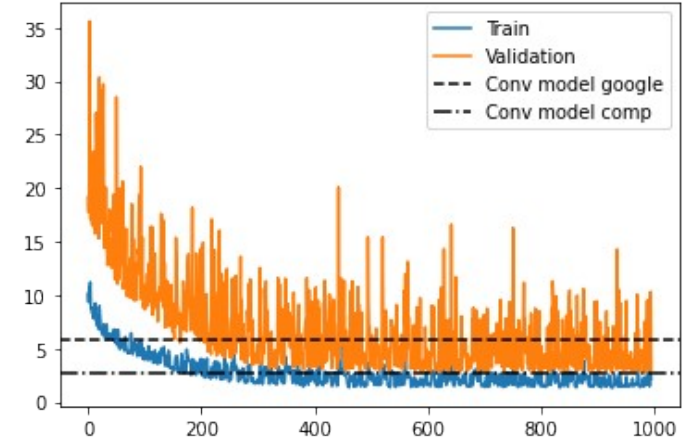
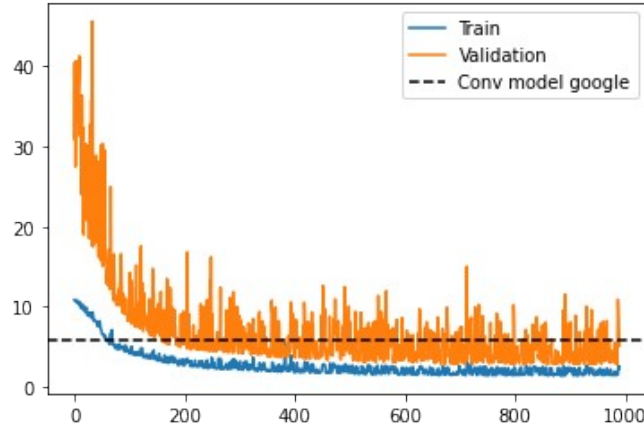
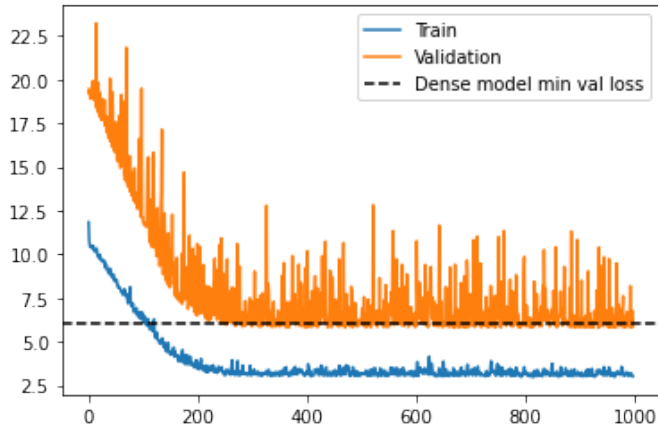
Vamos a ver que pasa en validación



RECETA

5) Hiperparámetros de entrenamiento: regulariza

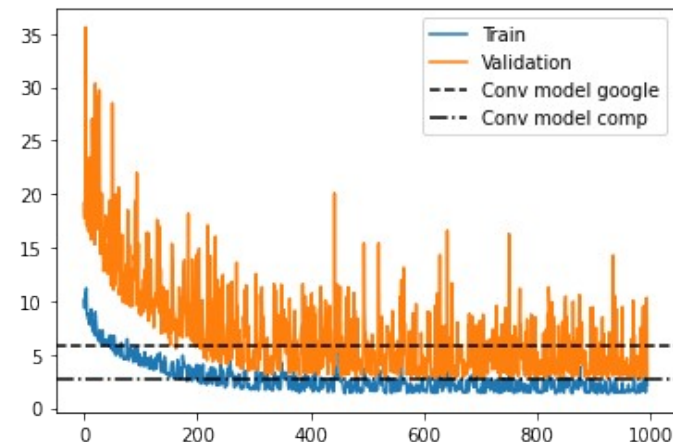
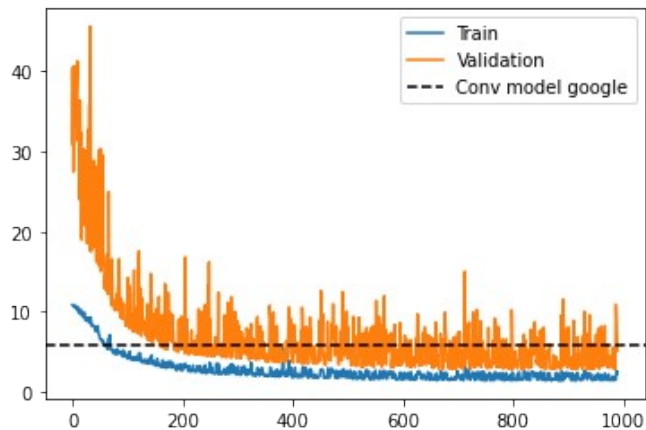
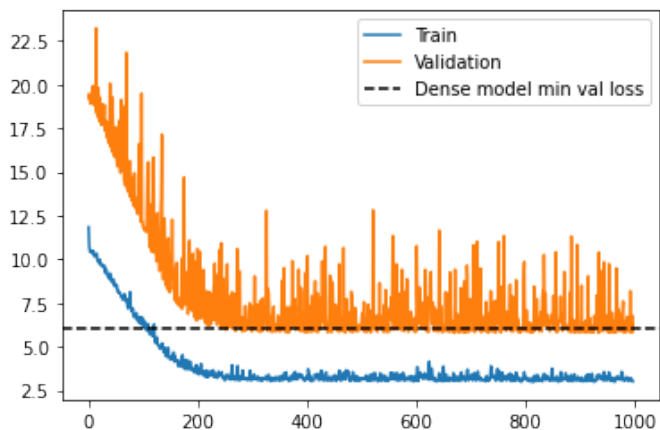
Vamos a ver que pasa en validación



RECETA

5) Hiperparámetros de entrenamiento: regulariza

Vamos a ver que pasa en validación



- **Reduce:** Dimensiones, Modelo y Batch size.
- **Técnicas:** Dropout, Weight decay, Batch normalization y Early stopping.
- A veces mejor un modelo mas grande (no preguntes).

RECETA



5) Hiperparámetros de entrenamiento

- Número de épocas
Las suficientes para que esté convergido claramente.
Es decir, la curva de validación no mejora.
- Regularización
La necesaria.
Que la curva de validación y de train estén pegaditas.
- Batch size
Tan grande como pueda.
Es el “causante principal” de la carga computacional.
- **Learning rate**

RECETA

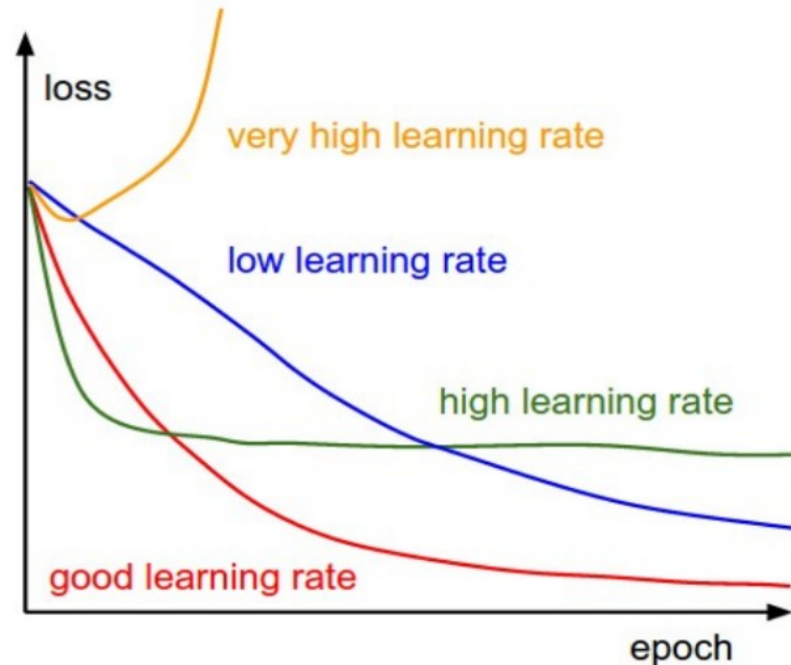
5) Hiperparámetros de entrenamiento: ajusta el learning rate

- Muy principio no hace falta muchas épocas

RECETA

5) Hiperparámetros de entrenamiento: ajusta el learning rate

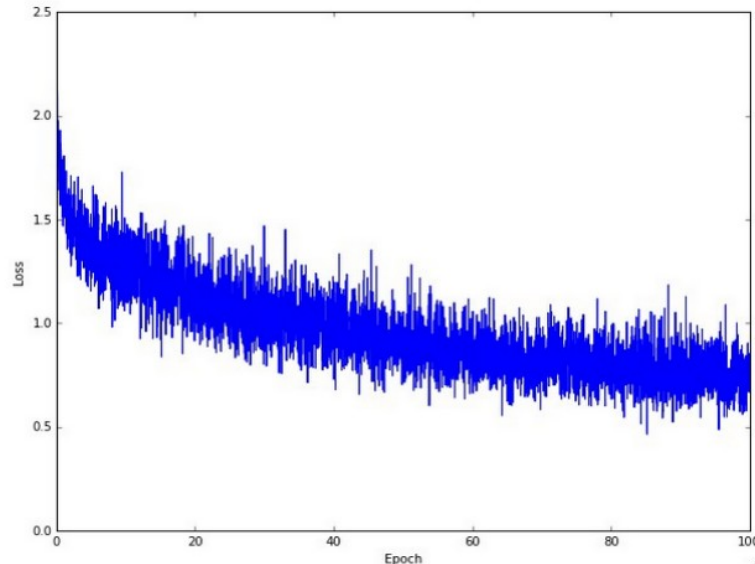
- Al principio no hace falta muchas épocas
- Prueba cambiando órdenes de magnitud (... , 10^3 , 10^2 , 10, 1, 0.1, ...)



RECETA

5) Hiperparámetros de entrenamiento: ajusta el learning rate

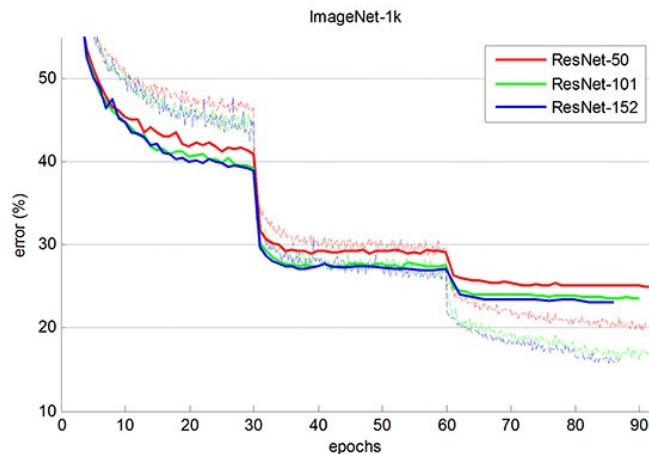
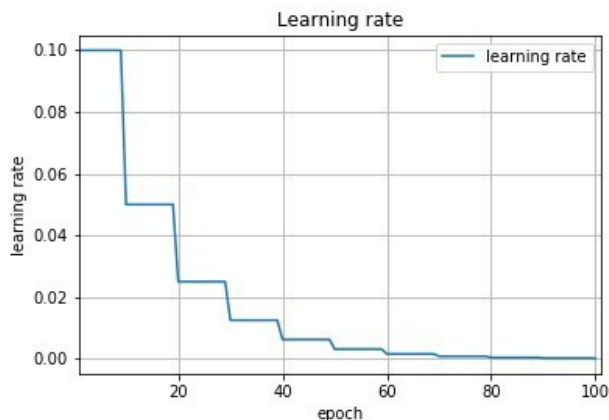
- Al principio no hace falta muchas épocas
- Prueba cambiando órdenes de magnitud (... , 10^3 , 10^2 , 10, 1, 0.1, ...)
- La vida real es ruidosa, suaviza tus curvas (tensorboard lo hace)



RECETA

5) Hiperparámetros de entrenamiento: ajusta el learning rate

- Al principio no hace falta muchas épocas
- Prueba cambiando órdenes de magnitud (... , 10^3 , 10^2 , 10, 1, 0.1, ...)
- La vida real es ruidosa, suaviza tus curvas (tensorboard lo hace)
- Reduce el learning rate mientras entrenas (cuando sature la curva)



RECETA

6) Entrenamiento final

- Fine-tuning.
 - Entrena mas tiempo
 - Busca de forma aleatoria en un grid.
Por ejemplo: Talos o Keras-tuner.
 - Optimización automática.
Hay cosas pero no muy fiables. Lo mejor es un (buen) becario ;-)
- Ensembles.



RECETA

6) TIPS

- Datos:

- Elige los datos mas informativos
- Decorrelaciona tus datos
- Aleatoriza tus datos

- Inicialización:

- Inicialización identidad
- Con los pesos de una red preentrenada
- Aleatoria Glorot

- Carga computacional.

- Lo que ocupa memoria es el backpropagation.
- Jugar con el tamaño del mini-batch, pero cuidado.
- Cuidado con el **overfitting**, sobre todo el “oculto”.
- El error de validación puede ser menor que el de train.
- Reentrenamiento de **la última capa** por separado.

RECETA

6) TIPS

- Callbacks.

- Usa early stopping
- Guarda el mejor modelo durante el entrenamiento.
- No uses LR schedule default al principio, luego sí.
- Hay distintos LR schedules

- Método de optimización:

- **Adam** con learning rate of $3e-4$.
- Prueba distintos LR.
- Estocastic gradient descent ayuda a regularizar

- Método de optimización (avanzado):

- Gradientes naturales
- Ajusta el learning rate de cada “peso” (sobre todo si son de naturaleza distinta)
- Las últimas capas entrenan mas rápido que las primeras (están mas cerca del error)

RECETA

6) TIPS

Usa redes preentrenadas

Tensorflow hub

<https://tfhub.dev/>

Tensorflow js

<https://www.tensorflow.org/js/models>

Tensorflow detection model zoo

https://github.com/tensorflow/models/blob/master/research/object_detection/g3doc/detection_model_zoo.md

Tensorflow models

<https://github.com/tensorflow/models/>

Hugging Face

<https://huggingface.co>

RECETA

6) TIPS

Buenos ingredientes

15 minutos + 1 fuego alto

2 agua + 1 arroz

Hervir varias veces

Prueba de sal justo antes del arroz

Hazla 100 veces

...



RECETA

Monitorizar. XAI (Explainable AI)

Visualiza a tope

- Datos de entrada de la red (después del preprocesado)
- Plotea las perdidas para todas las épocas y todos los datos.
- Las salidas.
- Los pesos.
- Los gradientes.
- Para cada muestra.
- ...

RECETA

Visualiza modelo. XAI (Explainable AI)

Karpathy

<https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

CNN explainer

<https://poloclub.github.io/cnn-explainer/>

CNN visualizer

<https://www.cs.ryerson.ca/~aharley/vis/conv/>

CNN Playground

<https://convnetplayground.fastforwardlabs.com/#/>

Microscope

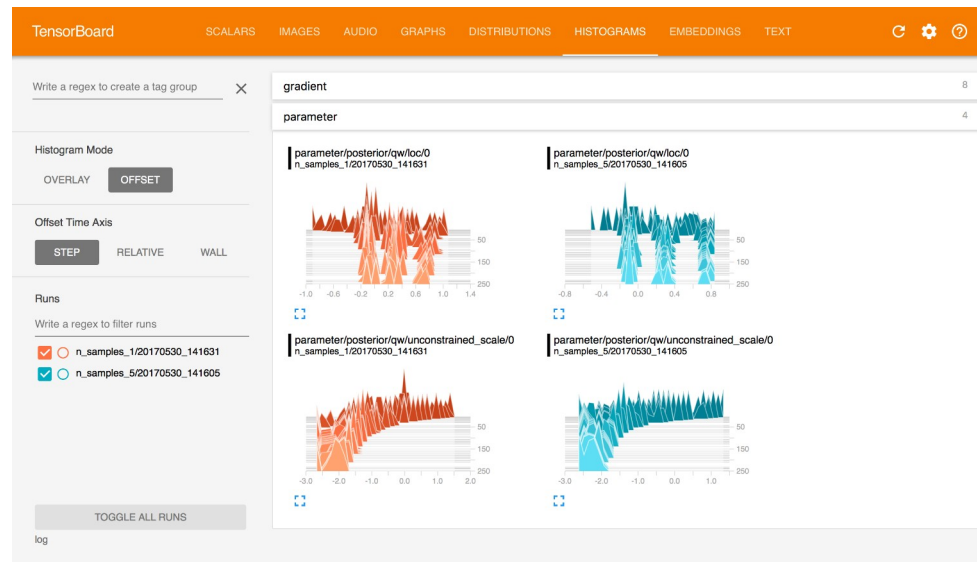
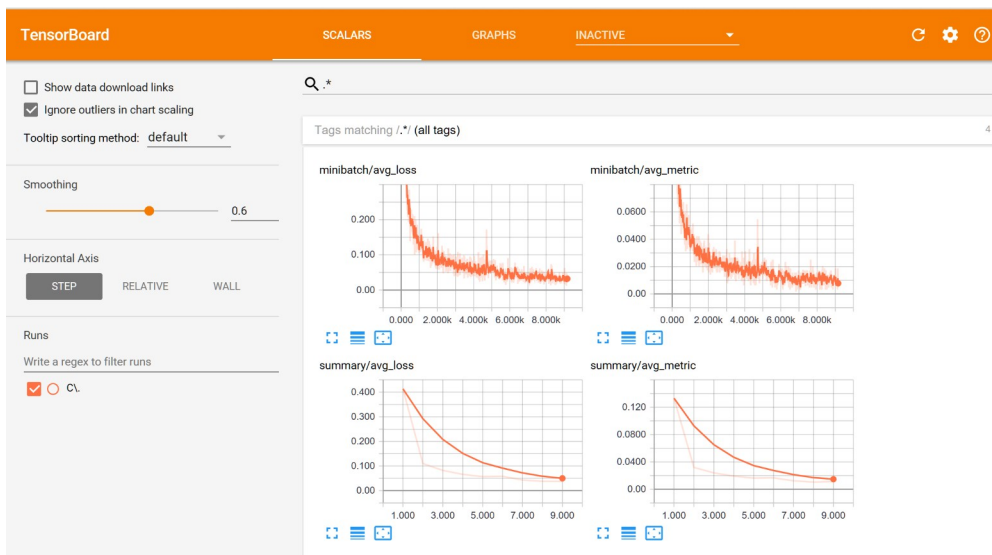
<https://microscope.openai.com/>

Lucid et al.

<https://github.com/tensorflow/lucid>

RECETA

Tensorboard

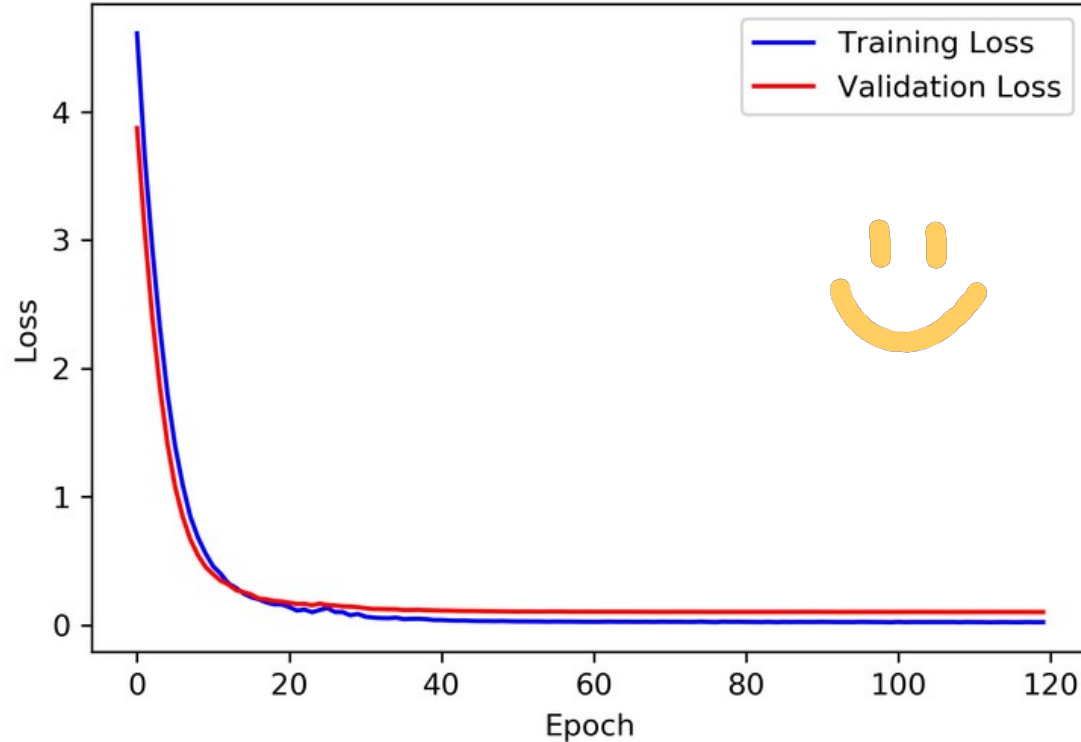


Weights and bias - 1dB

Curvas de aprendizaje

Curvas de aprendizaje

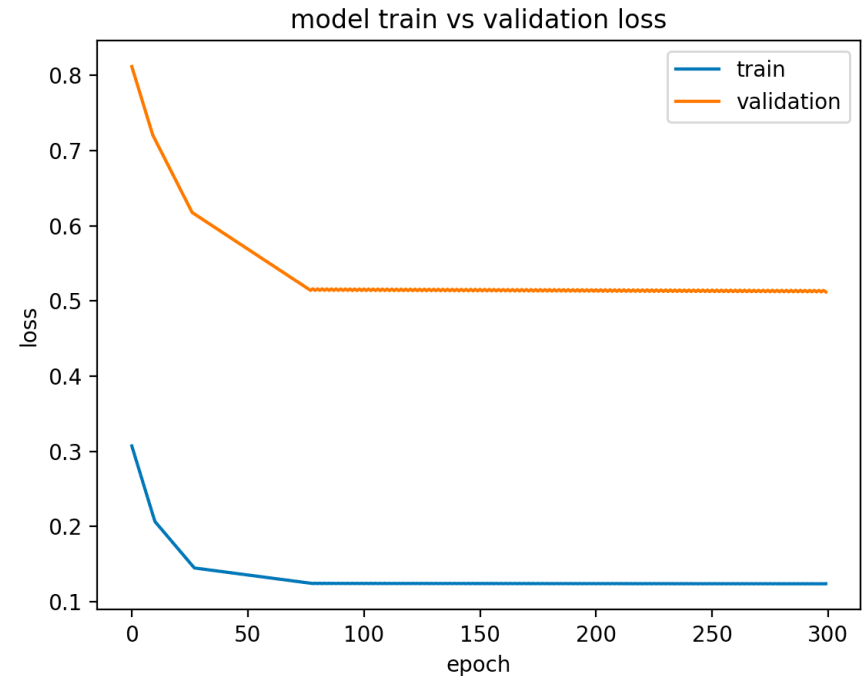
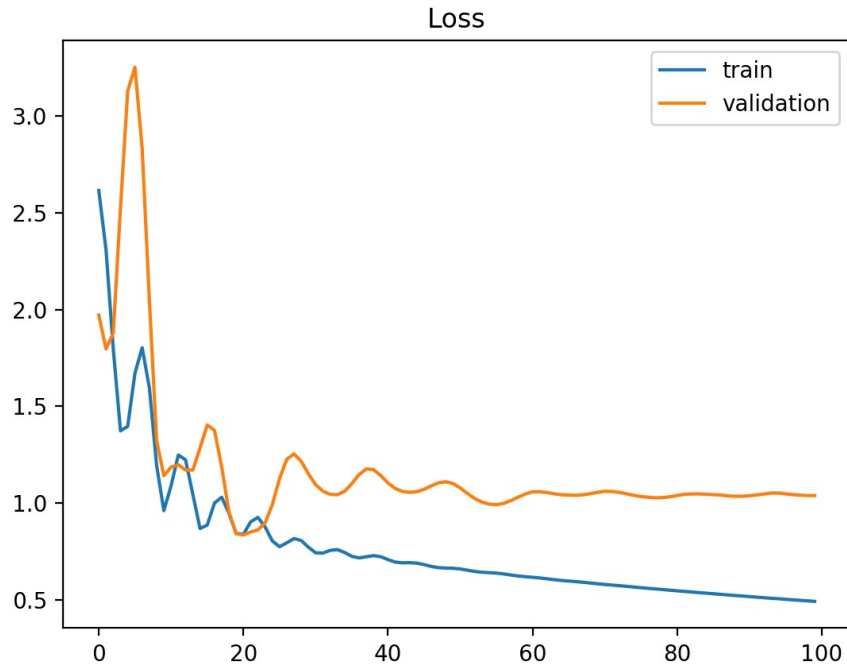
Curvas de aprendizaje: buen fitting



Curvas de aprendizaje

FALLOS DATASET

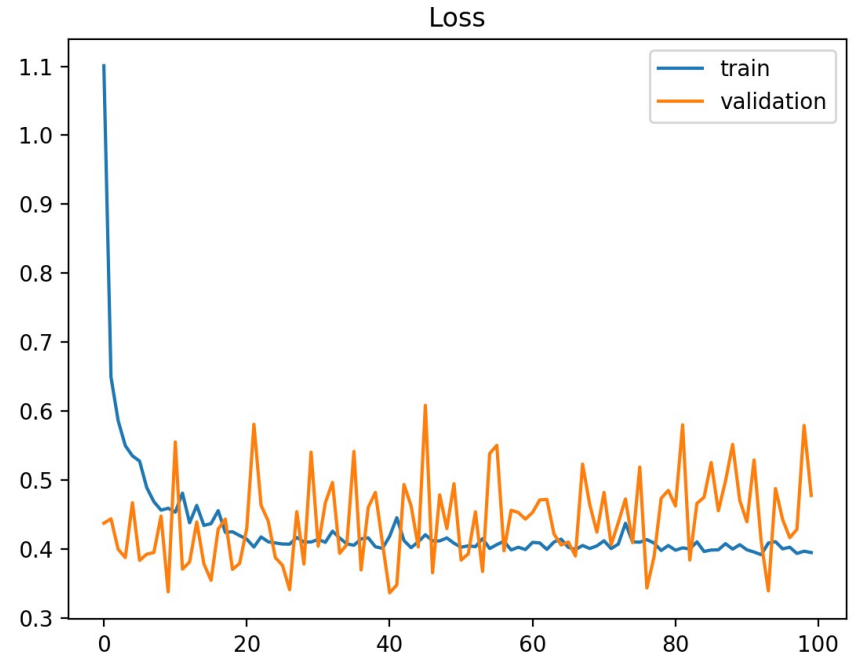
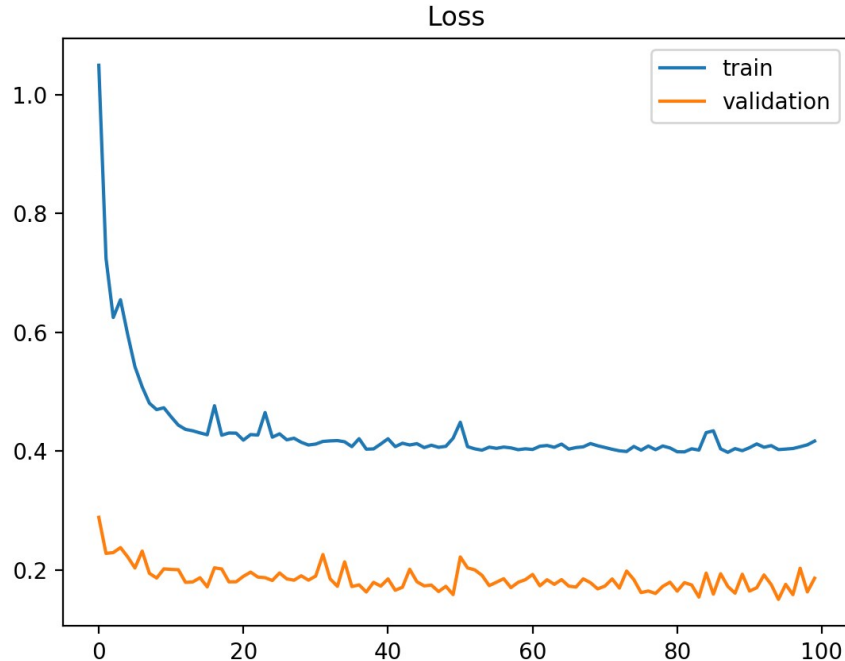
dataset de train/validación no representativos



Curvas de aprendizaje

FALLOS DATASET

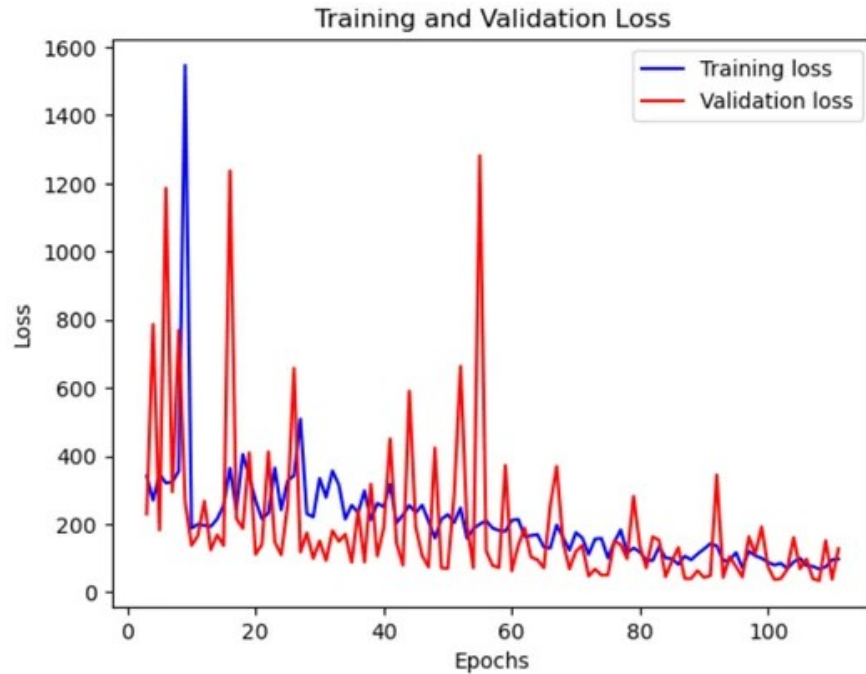
Datos de validación muy simples (poco representativos)



Curvas de aprendizaje

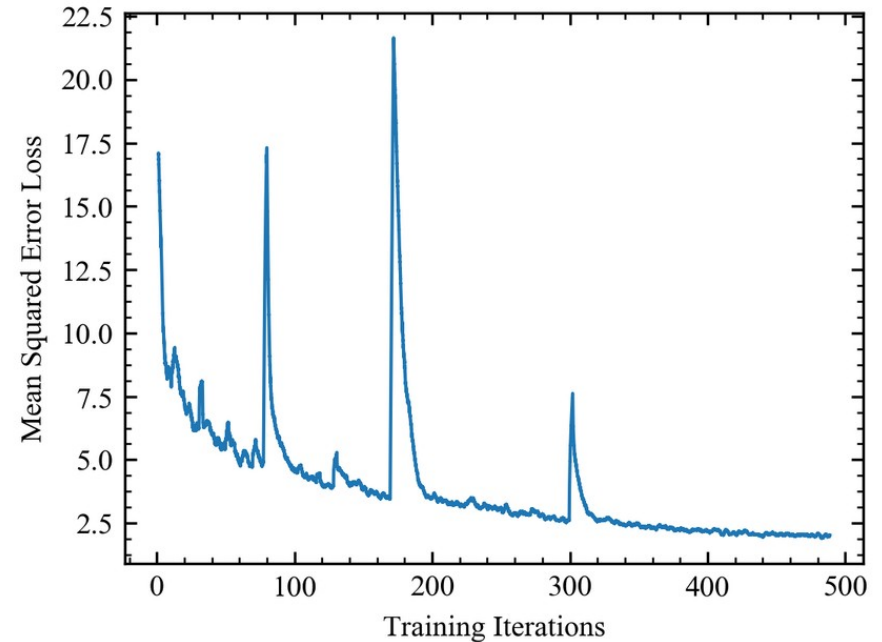
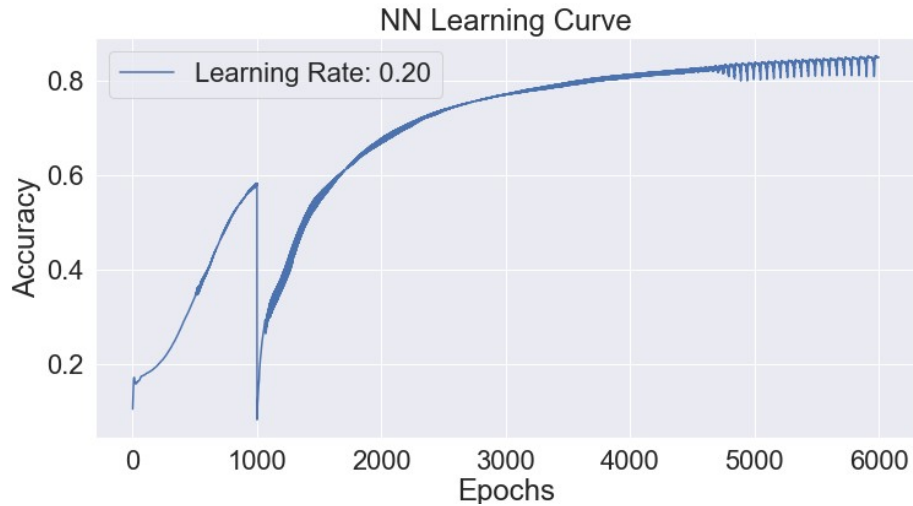
FALLOS DATASET

Pocos datos



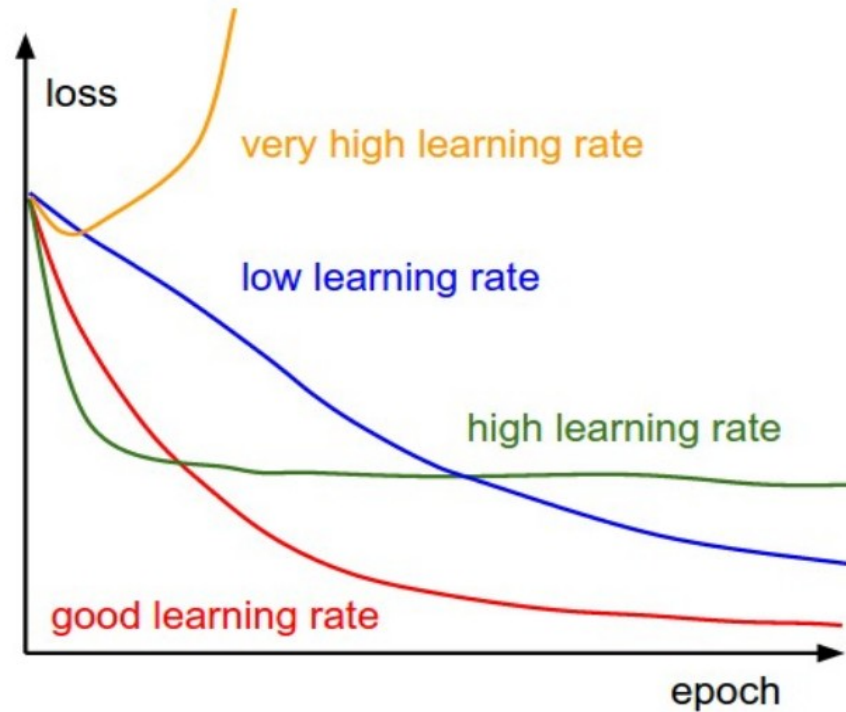
Curvas de aprendizaje

FALLOS OPTIMIZADOR
(o parámetros muy sensibles)



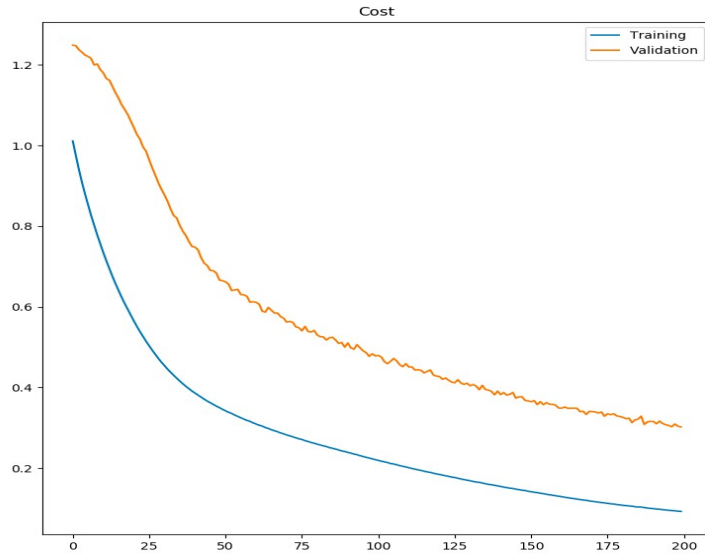
MONITORIZA

FALLOS Learning rate



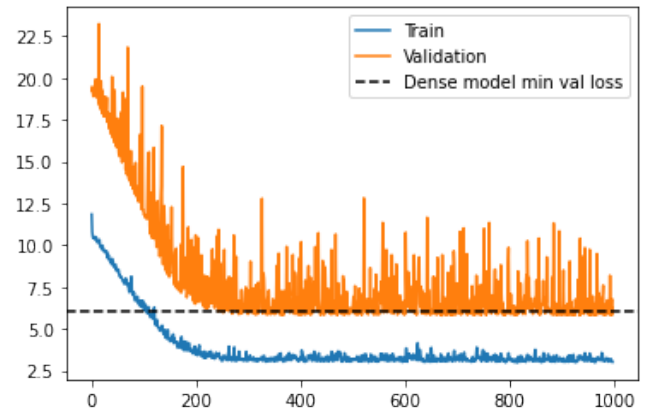
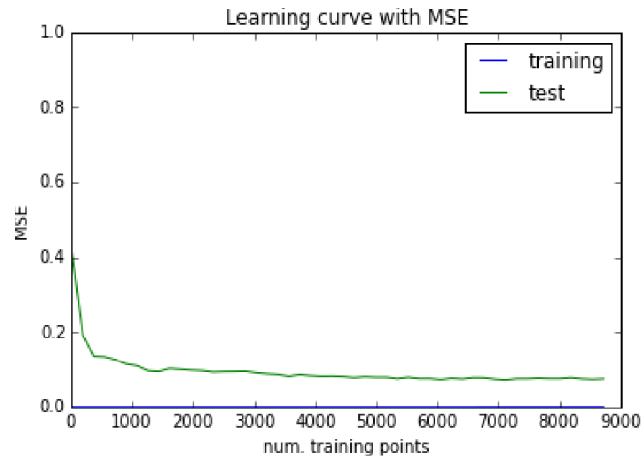
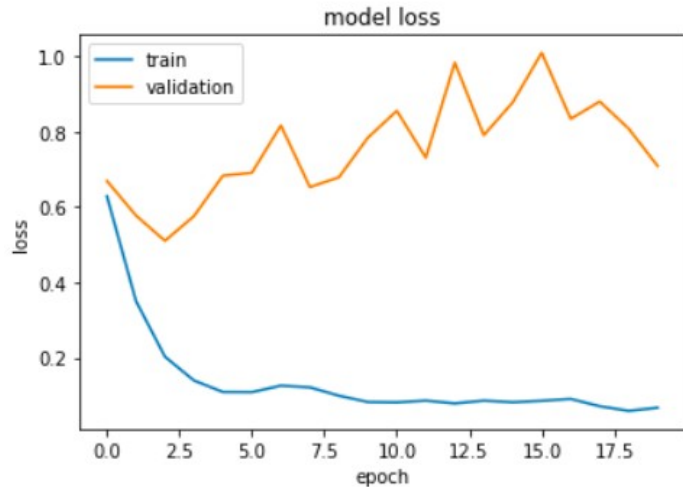
Curvas de aprendizaje

FALLOS Underfitting



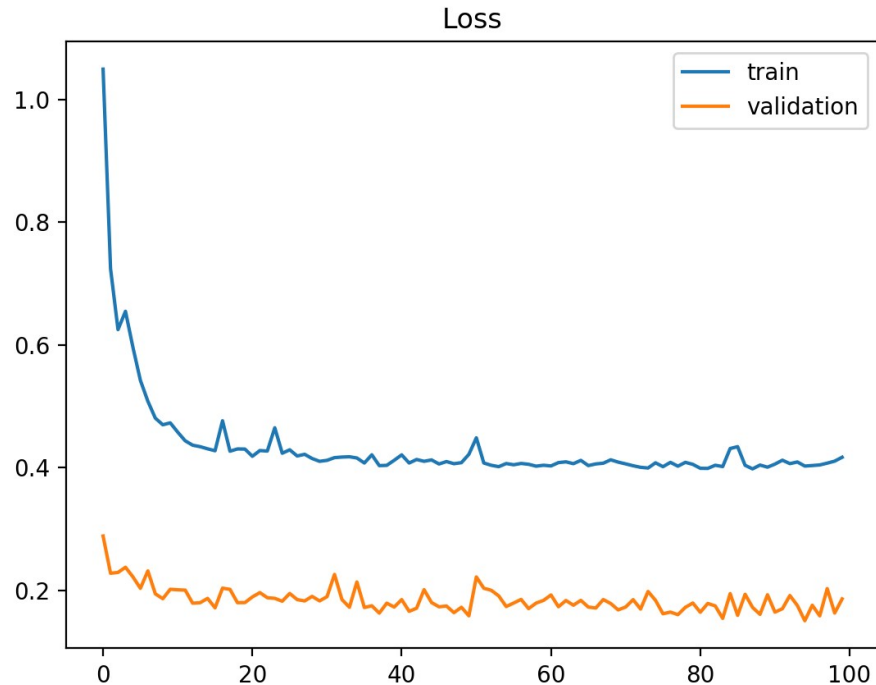
Curvas de aprendizaje

Curvas de aprendizaje: Overfitting



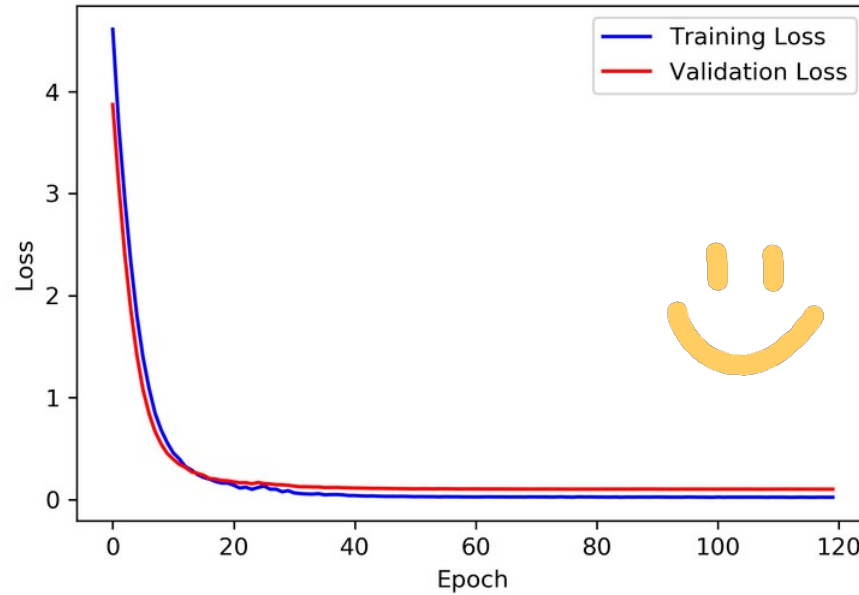
Curvas de aprendizaje

Curvas de aprendizaje: Sobrerregularizado



Curvas de aprendizaje

Curvas de aprendizaje: buen ajuste



OJO: Zoom + logaritmo en eje y